

M1 Informatique -Sorbonne Université

MU4IN801 - MLBDA

Modèles et Langages Bases de Données Avancées

Bernd Amann

COURS 4 – DONNÉES DU WEB
MODÈLE SEMI-STRUCTURÉ ET XML

Plan

Modèles de données semi-structurées

- Besoins
- Caractéristiques générales

OEM (Object Exchange Model)

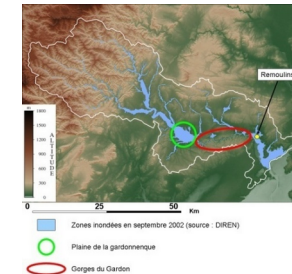
XML (Extensible Markup Language)

JSON (Javascript Object Notation)

Gestion de données avant



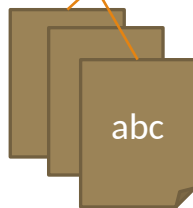
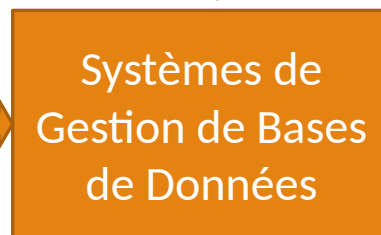
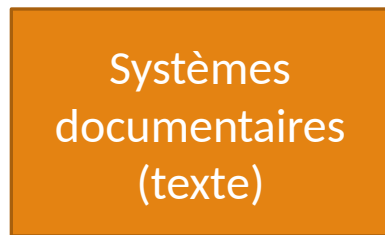
	A	B	C	D	E	F
	Prénom	Nom	Description	Quantité	Montant	Catégorie
1	John	Doe	100 kg de farine	10	1000	Aliments
2	Jane	Smith	50 kg de sucre	5	500	Aliments
3	Bob	Johnson	20 kg de riz	2	200	Aliments
4	Alice	Williams	10 kg de lentilles	1	100	Aliments
5	Charlie	Brown	5 kg de pois chiches	0.5	50	Aliments
6	Diana	Green	1 kg de haricots	0.1	10	Aliments
7	Eve	White	500 g de lentilles	0.5	50	Aliments
8	Frank	Black	250 g de pois chiches	0.25	25	Aliments
9	Grace	Grey	125 g de haricots	0.125	12.5	Aliments
10	Henry	Gold	62.5 g de lentilles	0.0625	6.25	Aliments
11	Ivy	Silver	31.25 g de pois chiches	0.03125	3.125	Aliments
12	Jack	Platinum	15.625 g de haricots	0.015625	1.5625	Aliments
13	Karen	Diamond	7.8125 g de lentilles	0.0078125	0.78125	Aliments
14	Liam	Emerald	3.90625 g de pois chiches	0.00390625	0.390625	Aliments
15	Mia	Sapphire	1.953125 g de haricots	0.001953125	0.1953125	Aliments
16	Noah	Ruby	0.9765625 g de lentilles	0.0009765625	0.09765625	Aliments
17	Olivia	Garnet	0.48828125 g de pois chiches	0.00048828125	0.048828125	Aliments
18	Peter	Amethyst	0.244140625 g de haricots	0.000244140625	0.0244140625	Aliments
19	Quinn	Peridot	0.1220703125 g de lentilles	0.0001220703125	0.01220703125	Aliments
20	Rachel	Topaz	0.06103515625 g de pois chiches	6.103515625e-05	0.006103515625	Aliments
21	Samuel	Malachite	0.030517578125 g de haricots	3.0517578125e-05	0.0030517578125	Aliments
22	Tina	Jade	0.0152587890625 g de lentilles	1.52587890625e-05	0.00152587890625	Aliments
23	Uma	Obsidian	0.00762939453125 g de pois chiches	7.62939453125e-06	0.000762939453125	Aliments
24	Victor	Flint	0.003814697265625 g de haricots	3.814697265625e-06	0.0003814697265625	Aliments
25	Wendy	Amber	0.0019073486328125 g de lentilles	1.9073486328125e-06	0.00019073486328125	Aliments
26	Xavier	Opal	0.00095367431640625 g de pois chiches	9.5367431640625e-07	9.5367431640625e-05	Aliments
27	Yara	Malachite	0.000476837158203125 g de haricots	4.76837158203125e-07	4.76837158203125e-05	Aliments
28	Zoe	Flint	0.0002384185791015625 g de lentilles	2.384185791015625e-07	2.384185791015625e-05	Aliments
29	Adam	Opal	0.00011920928955078125 g de pois chiches	1.1920928955078125e-07	1.1920928955078125e-05	Aliments
30	Bella	Amber	5.9604644775390625e-05 g de haricots	5.9604644775390625e-08	5.9604644775390625e-06	Aliments
31	Carl	Opal	2.9802322387695312e-05 g de lentilles	2.9802322387695312e-08	2.9802322387695312e-06	Aliments
32	Dora	Amber	1.4901161193847656e-05 g de pois chiches	1.4901161193847656e-08	1.4901161193847656e-06	Aliments
33	Ethan	Opal	7.450580596923828e-06 g de haricots	7.450580596923828e-09	7.450580596923828e-07	Aliments
34	Fiona	Amber	3.725290298461914e-06 g de lentilles	3.725290298461914e-09	3.725290298461914e-07	Aliments
35	George	Opal	1.862645149230957e-06 g de pois chiches	1.862645149230957e-09	1.862645149230957e-07	Aliments
36	Hannah	Amber	9.313225746154785e-07 g de haricots	9.313225746154785e-10	9.313225746154785e-08	Aliments
37	Ian	Opal	4.656612873077392e-07 g de lentilles	4.656612873077392e-10	4.656612873077392e-08	Aliments
38	Jane	Amber	2.328306436538696e-07 g de pois chiches	2.328306436538696e-10	2.328306436538696e-08	Aliments
39	Karen	Opal	1.164153218269348e-07 g de haricots	1.164153218269348e-10	1.164153218269348e-08	Aliments
40	Liam	Amber	5.82076609134674e-08 g de lentilles	5.82076609134674e-11	5.82076609134674e-09	Aliments
41	Mia	Opal	2.91038304567337e-08 g de pois chiches	2.91038304567337e-11	2.91038304567337e-09	Aliments
42	Noah	Amber	1.455191522836685e-08 g de haricots	1.455191522836685e-11	1.455191522836685e-09	Aliments
43	Olivia	Opal	7.275957614183425e-09 g de lentilles	7.275957614183425e-12	7.275957614183425e-10	Aliments
44	Peter	Amber	3.637978807091712e-09 g de pois chiches	3.637978807091712e-12	3.637978807091712e-10	Aliments
45	Quinn	Opal	1.818989403545856e-09 g de haricots	1.818989403545856e-12	1.818989403545856e-10	Aliments
46	Rachel	Amber	9.09494701772928e-10 g de lentilles	9.09494701772928e-13	9.09494701772928e-11	Aliments
47	Samuel	Opal	4.54747350886464e-10 g de pois chiches	4.54747350886464e-13	4.54747350886464e-11	Aliments
48	Tina	Amber	2.27373675443232e-10 g de haricots	2.27373675443232e-13	2.27373675443232e-11	Aliments
49	Uma	Opal	1.13686837721616e-10 g de lentilles	1.13686837721616e-13	1.13686837721616e-11	Aliments
50	Victor	Amber	5.6843418860808e-11 g de pois chiches	5.6843418860808e-14	5.6843418860808e-12	Aliments
51	Wendy	Opal	2.8421709430404e-11 g de haricots	2.8421709430404e-14	2.8421709430404e-12	Aliments
52	Xavier	Amber	1.4210854715202e-11 g de lentilles	1.4210854715202e-14	1.4210854715202e-12	Aliments
53	Yara	Opal	7.105427357601e-12 g de pois chiches	7.105427357601e-15	7.105427357601e-13	Aliments
54	Zoe	Amber	3.5527136788005e-12 g de haricots	3.5527136788005e-15	3.5527136788005e-13	Aliments
55	Adam	Opal	1.77635683940025e-12 g de lentilles	1.77635683940025e-15	1.77635683940025e-13	Aliments
56	Bella	Amber	8.88178419700125e-13 g de pois chiches	8.88178419700125e-16	8.88178419700125e-14	Aliments
57	Carl	Opal	4.440892098500625e-13 g de haricots	4.440892098500625e-16	4.440892098500625e-14	Aliments
58	Dora	Amber	2.2204460492503125e-13 g de lentilles	2.2204460492503125e-16	2.2204460492503125e-14	Aliments
59	Ethan	Opal	1.1102230246251562e-13 g de pois chiches	1.1102230246251562e-16	1.1102230246251562e-14	Aliments
60	Fiona	Amber	5.551115123125781e-14 g de haricots	5.551115123125781e-17	5.551115123125781e-15	Aliments
61	George	Opal	2.7755575615628905e-14 g de lentilles	2.7755575615628905e-17	2.7755575615628905e-15	Aliments
62	Hannah	Amber	1.3877787807814452e-14 g de pois chiches	1.3877787807814452e-17	1.3877787807814452e-15	Aliments
63	Ian	Opal	6.938893903907226e-15 g de haricots	6.938893903907226e-18	6.938893903907226e-16	Aliments
64	Jane	Amber	3.469446951953613e-15 g de lentilles	3.469446951953613e-18	3.469446951953613e-16	Aliments
65	Karen	Opal	1.7347234759768065e-15 g de pois chiches	1.7347234759768065e-18	1.7347234759768065e-16	Aliments
66	Liam	Amber	8.673617379884032e-16 g de haricots	8.673617379884032e-19	8.673617379884032e-17	Aliments
67	Mia	Opal	4.336808689942016e-16 g de lentilles	4.336808689942016e-19	4.336808689942016e-17	Aliments
68	Noah	Amber	2.168404344971008e-16 g de pois chiches	2.168404344971008e-19	2.168404344971008e-17	Aliments
69	Olivia	Opal	1.084202172485504e-16 g de haricots	1.084202172485504e-19	1.084202172485504e-17	Aliments
70	Peter	Amber	5.42101086242752e-17 g de lentilles	5.42101086242752e-20	5.42101086242752e-18	Aliments
71	Quinn	Opal	2.71050543121376e-17 g de pois chiches	2.71050543121376e-20	2.71050543121376e-18	Aliments
72	Rachel	Amber	1.35525271560688e-17 g de haricots	1.35525271560688e-20	1.35525271560688e-18	Aliments
73	Samuel	Opal	6.7762635780344e-18 g de lentilles	6.7762635780344e-21	6.7762635780344e-19	Aliments
74	Tina	Amber	3.3881317890172e-18 g de pois chiches	3.3881317890172e-21	3.3881317890172e-19	Aliments
75	Uma	Opal	1.6940658945086e-18 g de haricots	1.6940658945086e-21	1.6940658945086e-19	Aliments
76	Victor	Amber	8.470329472543e-19 g de lentilles	8.470329472543e-22	8.470329472543e-20	Aliments
77	Wendy	Opal	4.2351647362715e-19 g de pois chiches	4.2351647362715e-22	4.2351647362715e-20	Aliments
78	Xavier	Amber	2.11758236813575e-19 g de haricots	2.11758236813575e-22	2.11758236813575e-20	Aliments
79	Yara	Opal	1.058791184067875e-19 g de lentilles	1.058791184067875e-22	1.058791184067875e-20	Aliments
80	Zoe	Amber	5.293955920339375e-20 g de pois chiches	5.293955920339375e-23	5.293955920339375e-21	Aliments
81	Adam	Opal	2.6469779601696875e-20 g de haricots	2.6469779601696875e-23	2.6469779601696875e-21	Aliments
82	Bella	Amber	1.3234889800848437e-20 g de lentilles	1.3234889800848437e-23	1.3234889800848437e-21	Aliments
83	Carl	Opal	6.617444900424219e-21 g de pois chiches	6.617444900424219e-24	6.617444900424219e-22	Aliments
84	Dora	Amber	3.3087224502121095e-21 g de haricots	3.3087224502121095e-24	3.3087224502121095e-22	Aliments
85	Ethan	Opal	1.6543612251060547e-21 g de lentilles	1.6543612251060547e-24	1.6543612251060547e-22	Aliments
86	Fiona	Amber	8.271806125530274e-22 g de pois chiches	8.271806125530274e-25	8.271806125530274e-23	Aliments
87	George	Opal	4.135903062765137e-22 g de haricots	4.135903062765137e-25	4.135903062765137e-23	Aliments
88	Hannah	Amber	2.0679515313825685e-22 g de lentilles	2.0679515313825685e-25	2.0679515313825685e-23	Aliments
89	Ian	Opal	1.0339757656912842e-22 g de pois chiches	1.0339757656912842e-25	1.0339757656912842e-23	Aliments
90	Jane	Amber	5.169878828456421e-23 g de haricots	5.169878828456421e-26	5.169878828456421e-24	Aliments
91	Karen	Opal	2.5849394142282105e-23 g de lentilles	2.5849394142282105e-26	2.5849394142282105e-24	Aliments
92	Liam	Amber	1.2924697071141052e-23 g de pois chiches	1.2924697071141052e-26	1.2924697071141052e-24	Aliments
93	Mia	Opal	6.462348535570526e-24 g de haricots	6.462348535570526e-27	6.462348535570526e-25	Aliments
94	Noah	Amber	3.231174267785263e-24 g de lentilles	3.231174267785263e-27	3.231174267785263e-25	Aliments
95	Olivia	Opal	1.6155871338926315e-24 g de pois chiches	1.6155871338926315e-27	1.6155871338926315e-25	Aliments
96	Peter	Amber	8.077935669463157e-25 g de haricots	8.077935669463157e-28	8.077935669463157e-26	Aliments
97	Quinn	Opal	4.0389678347315785e-25 g de lentilles	4.0389678347315785e-28	4.0389678347315785e-26	Aliments
98	Rachel	Amber	2.0194839173657892e-25 g de pois chiches	2.0194839173657892e-28	2.0194839173657892e-26	Aliments
99	Samuel	Opal	1.0097419586828946e-25 g de haricots	1.0097419586828946e-28	1.0097419586828946e-26	Aliments
100	Tina	Amber	5.048709793414473e-26 g de lentilles	5.048709793414473e-29	5.048709793414473e-27	Aliments



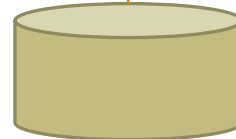
Recherche
d'information

Requêtes
Transactions

Métadonnées



texte

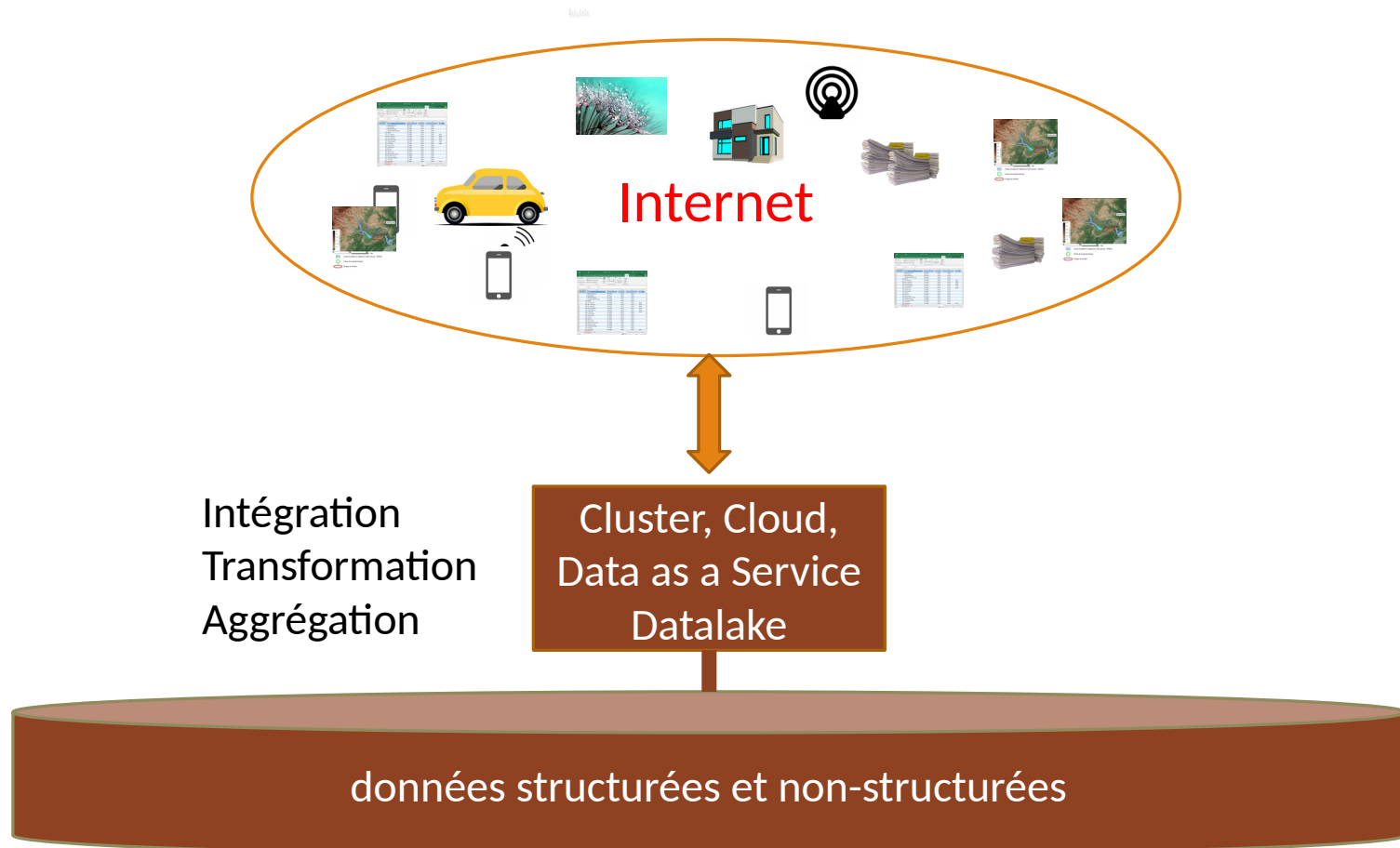


données structurées



audio, images, vidéos

Gestion de données aujourd'hui



Quel modèle de données ?

Utiliser les modèles de données structurées (relationnel, relationnel-objet) pose des problèmes :

- La structure est **trop rigide**
- les données peuvent ne pas être conformes au schéma
- **l'évolution** de la structure de données conduit à des évolutions de schéma difficiles à mettre en œuvre.

Nécessité d'un modèle général et **souple, sans trop de contraintes**, avec un *langage de requêtes* associé
⇒ *modèles semi-structurés*

Caractéristiques des modèles de données semi-structurées

DONNÉES

Structure irrégulière :

- Ex: adresse sous forme de nuplet avec et sans codes postaux et sous forme de chaîne de caractères

Structure « implicite » :

- Ex: documents textuels avec des sections et sous-sections

Données partielles / incomplètes :

- Ex: adresses avec et sans numéros de rues

SCHÉMAS

Schéma descriptifs

- Description et validation de données déjà existantes
- Plusieurs schémas possibles pour les mêmes données (précision versus taille)

Schéma complexes et volumineuses

- Ex: données irrégulières: taille données ~ taille schéma

Schémas évolutifs

- Ex: intégration de données ⇒ modification du schéma

REQUÊTES

Requêtes « flexibles »:

- Interrogation de données et de schémas

Requêtes approximatives

- Ignorer le schéma
- Interroger avec connaissance partielle de la structure

Modèle de données semi-structurées

Principe :

- partir des données et trouver une structure commune, souple.

Les modèles semi-structurés utilisent des graphes étiquetés pour représenter les données.

Les modèles diffèrent par

- le *rôle des étiquettes* : type, valeur, identifiant, ...
- l'endroit des étiquettes : arêtes et/ou nœuds
- l'existence ou non d'un ordonnancement
- La représentation du partage d'information (références)

OEM (Object Exchange Model)

Bases de données OEM = objet

Chaque objet a

- une **étiquette** (chaîne de caractères)
- un **identificateur**
- une **valeur** avec son **type** (*optionnels*).

Objets atomiques

- valeurs atomique : integer, real, string, html, audio, java, etc.

Objets complexe:

- valeur = **collection d'identificateurs** (références) : set, list

personne &1 set

Valeur de &1

nom &32 string "Jean Martin "

age &33 integer 25

adresse &3 set

rue &35 string "97 rue Duval"

ville &34 string "Paris"

Représentation graphique

personne &1 set

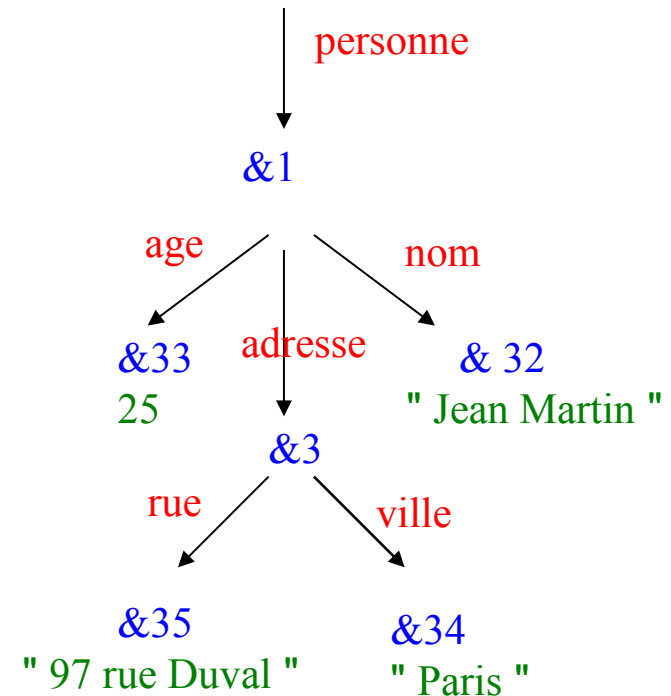
nom &32 string "Jean Martin "

age &33 integer 25

adresse &3 set

rue &35 string "97 rue Duval"

ville &34 string "Paris"



Partage d'objets

annuaire &50

personne &1

nom &32 "Jean Martin "

age &33 25

adresse &3

rue &35 "97 rue Duval"

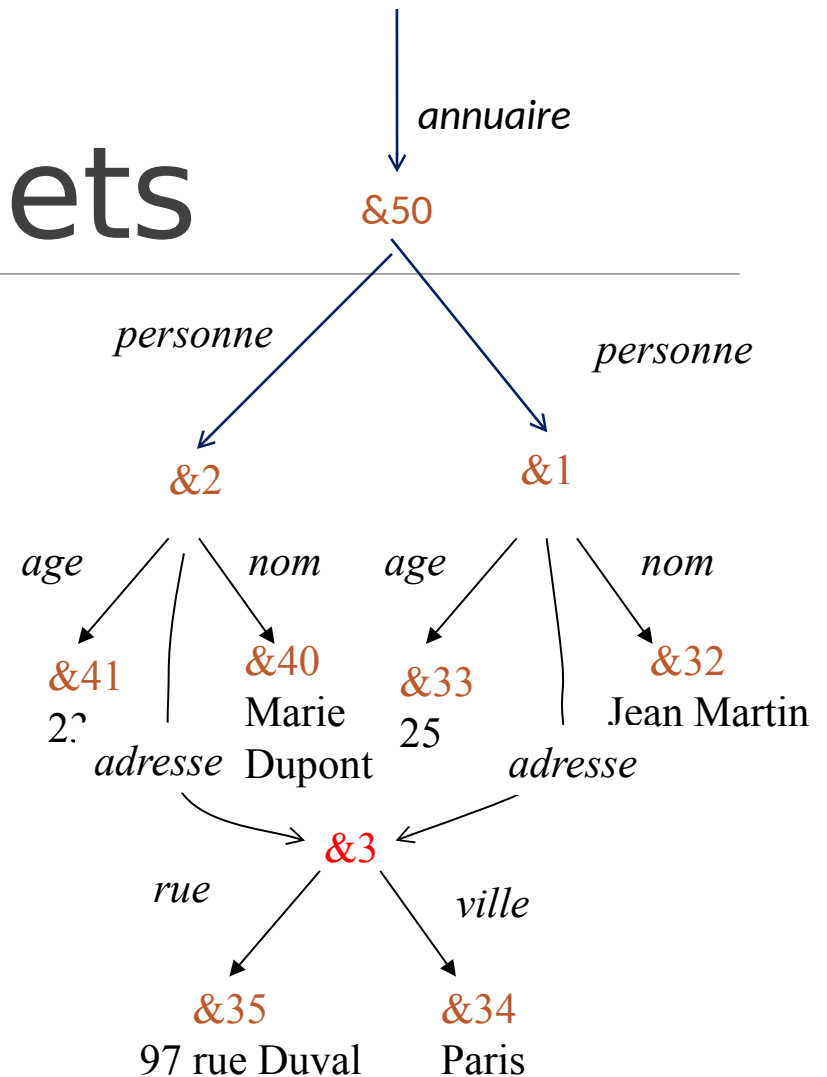
ville &34 "Paris"

personne &2

nom &40 "Marie Dupont"

age &41 23

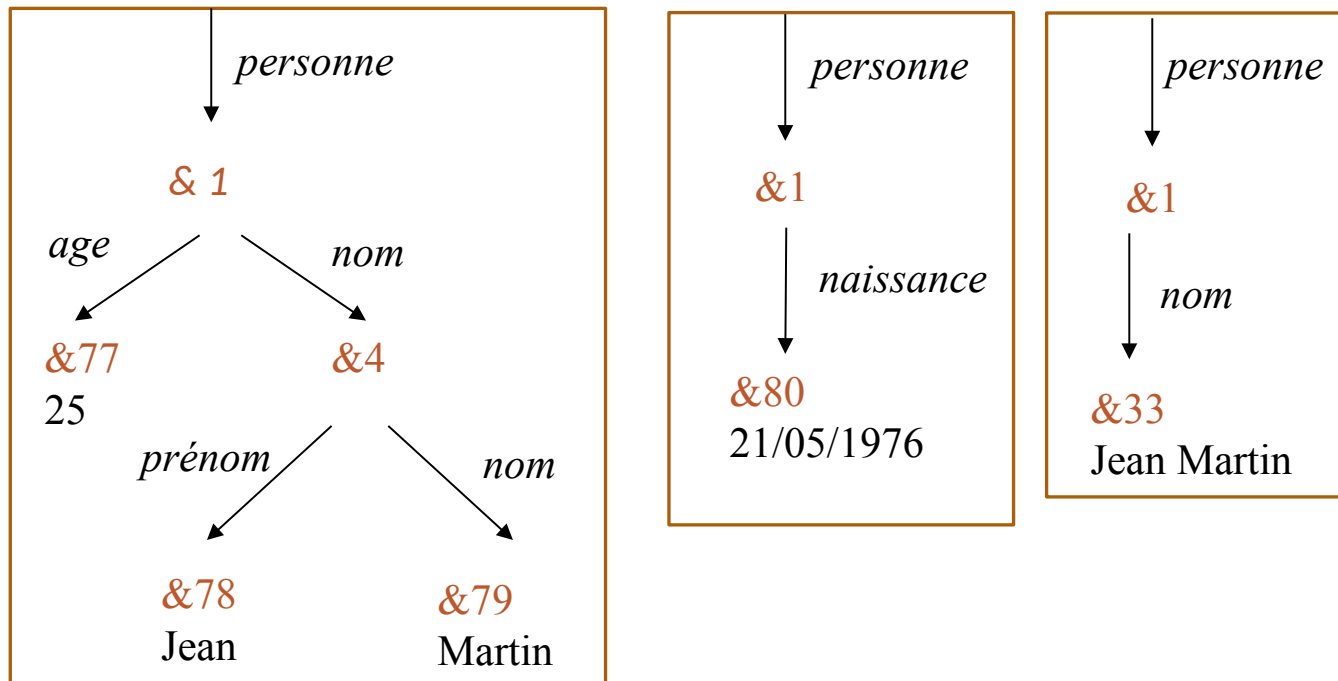
adresse &3



Souplesse de structuration

Co-existence de structures diverses pour les mêmes types d'entités

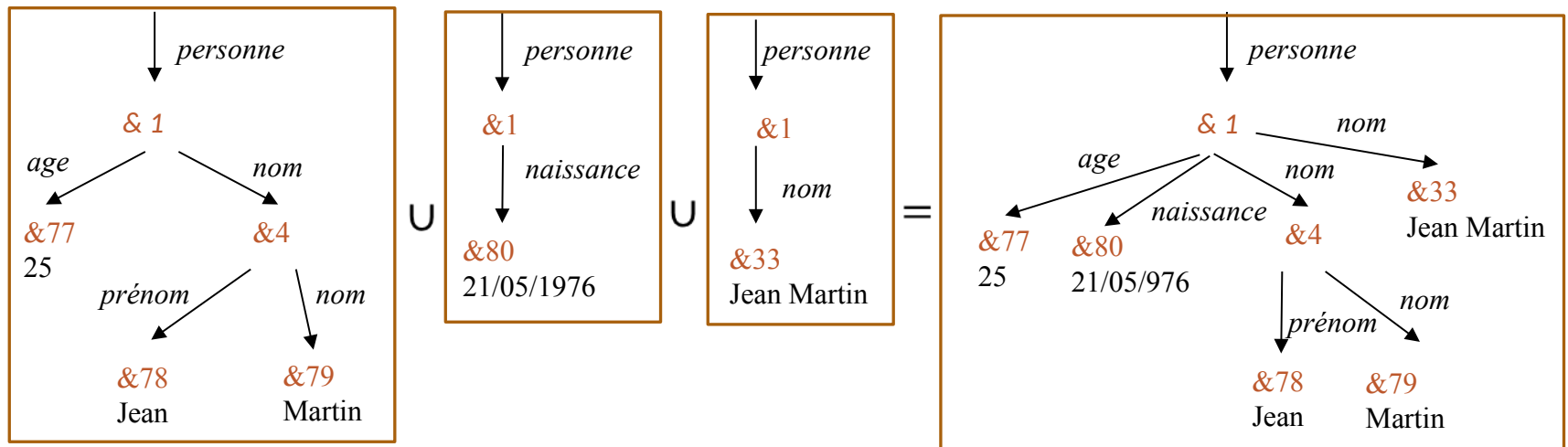
Exemple : Jean Martin



Integration « facile »

Co-existence de structures diverses pour les mêmes types d'entités

Exemple : **personne**



Langages de requêtes semi-structurés

Les langages classiques ne sont plus appropriés:

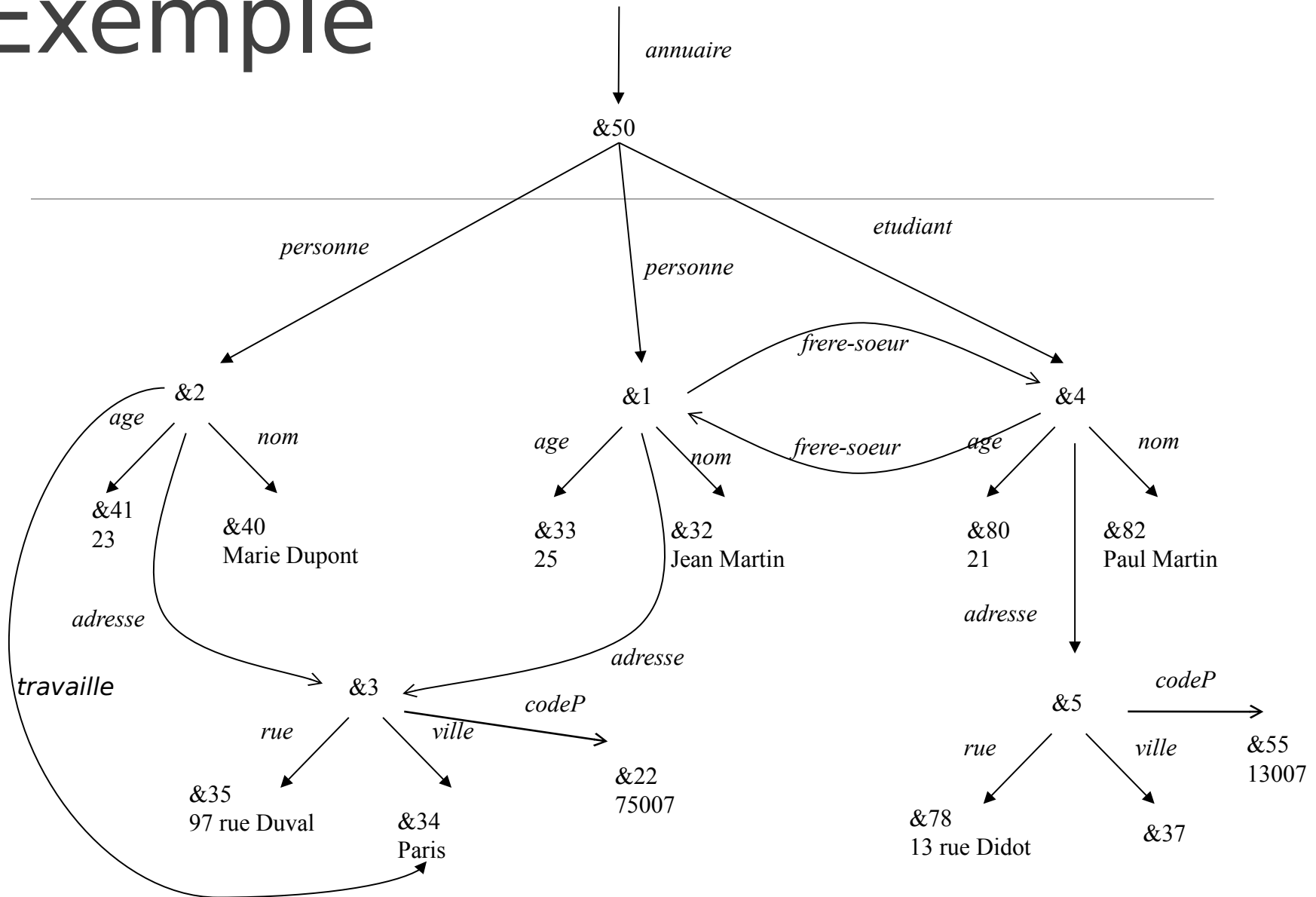
- la structure exacte des données n'est pas connue
- la structure est irrégulière
- des concepts similaires sont représentés différemment
- ensembles hétérogènes

Le langage doit :

- s'adapter à la structure souple, irrégulière, mouvante des données
- pouvoir interroger sans connaître la structure exacte des données
- rester proche du style SQL/OQL (langage déclaratif simple)

⇒ interrogation *navigationnelle* qui s'appuie sur des *expressions de chemins*.

Exemple



Représentation textuelle

annuaire &50

Personne &1

nom &32 "Jean Martin"

age &33 25

frere-soeur &4

adresse &3

rue &35 "97 rue Duval"

ville &34 "Paris"

codeP &22 75007

Personne &2

nom &40 "Marie Dupont"

age &41 23

adresse &3

travaille &34

Etudiant &4

nom &82 "Paul Martin"

age &80 21

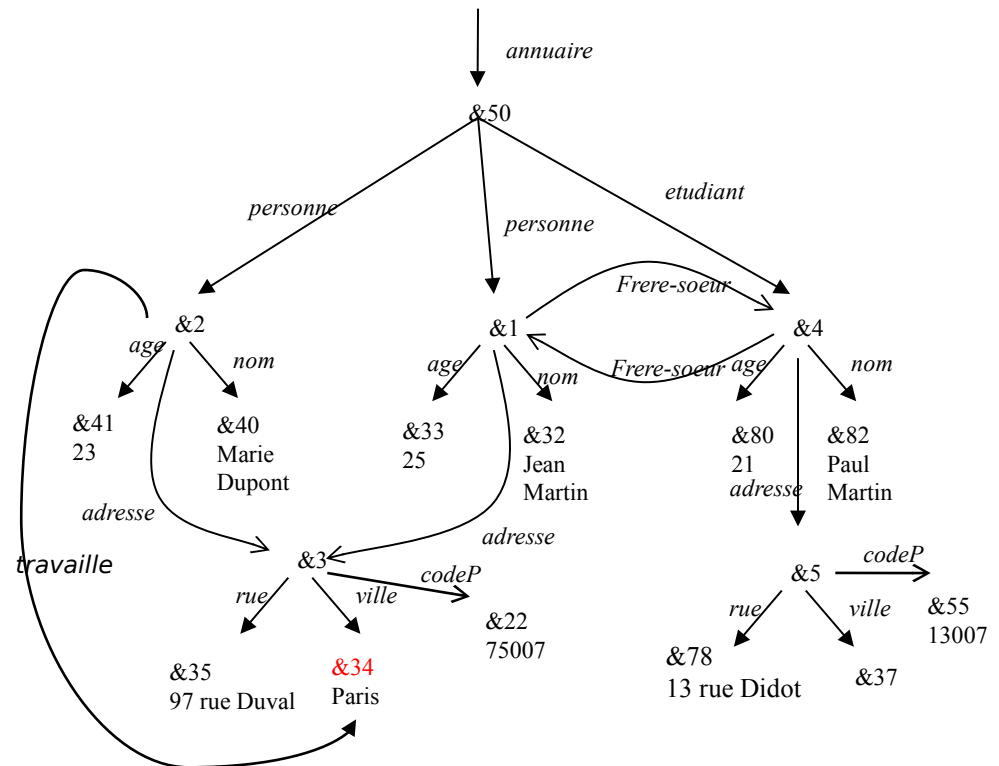
frere-soeur &1

adresse &5

rue &78 "13 rue Didot"

ville &37

codeP &55 75014



Expression de chemin

Expression de chemin (suite d'étiquettes) :

- Une expression de chemin est une séquence $Z.l_1.l_2 \dots l_n$, où les l_i sont des étiquettes et Z est un objet (ou une variable dénotant un objet).

Exemple : annuaire.nom.adresse

Chemin de données :

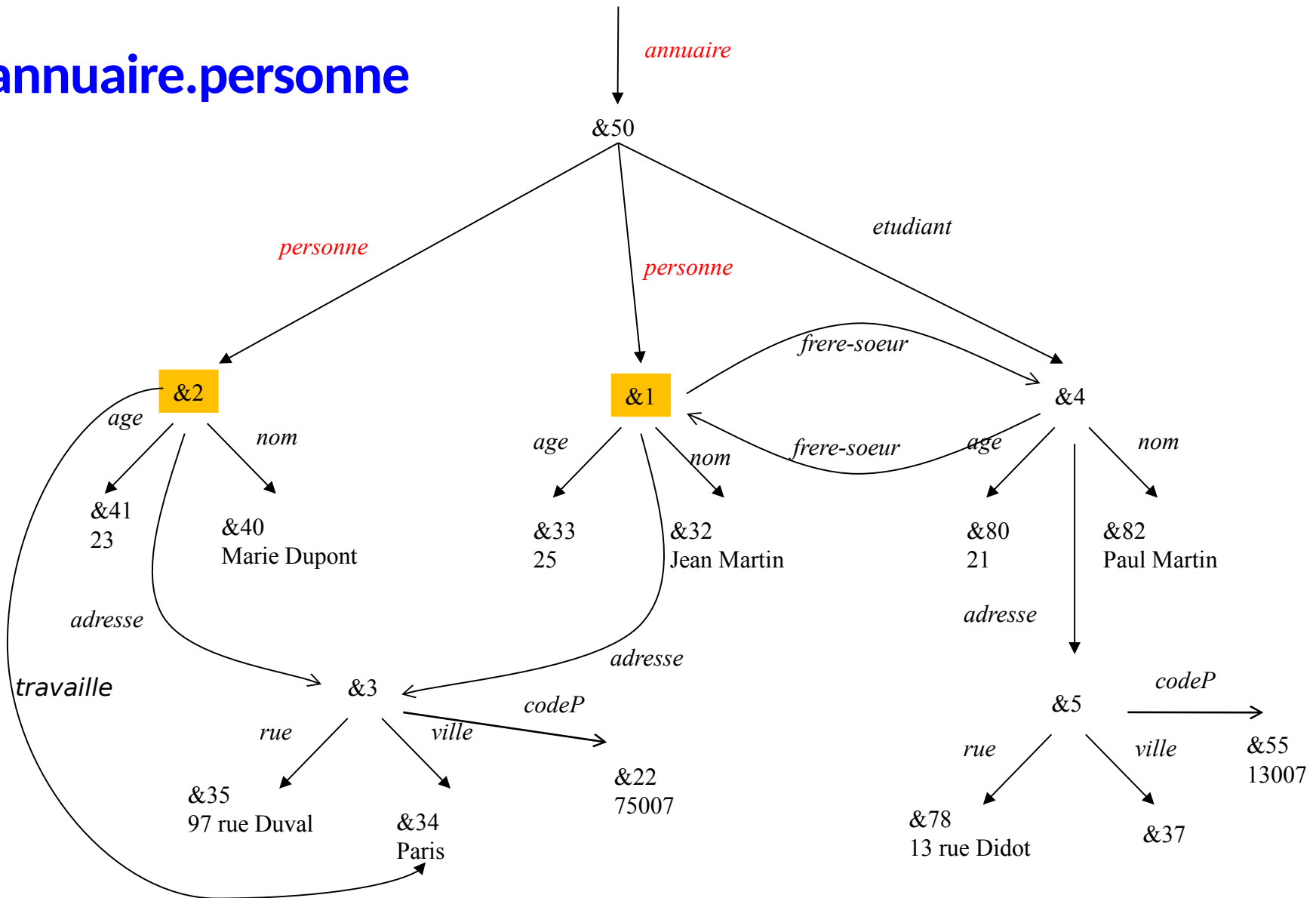
- Un chemin de données est une séquence $O_0, l_1, O_1, \dots, l_n, O_n$, où les O_i sont des objets, et pour chaque i , il existe un arc étiqueté l_i entre O_{i-1} et O_i .

Partant d'un objet $Z = O_0$, il peut exister plusieurs chemins de données correspondant à l'expression $Z.l_1.l_2 \dots l_n$.

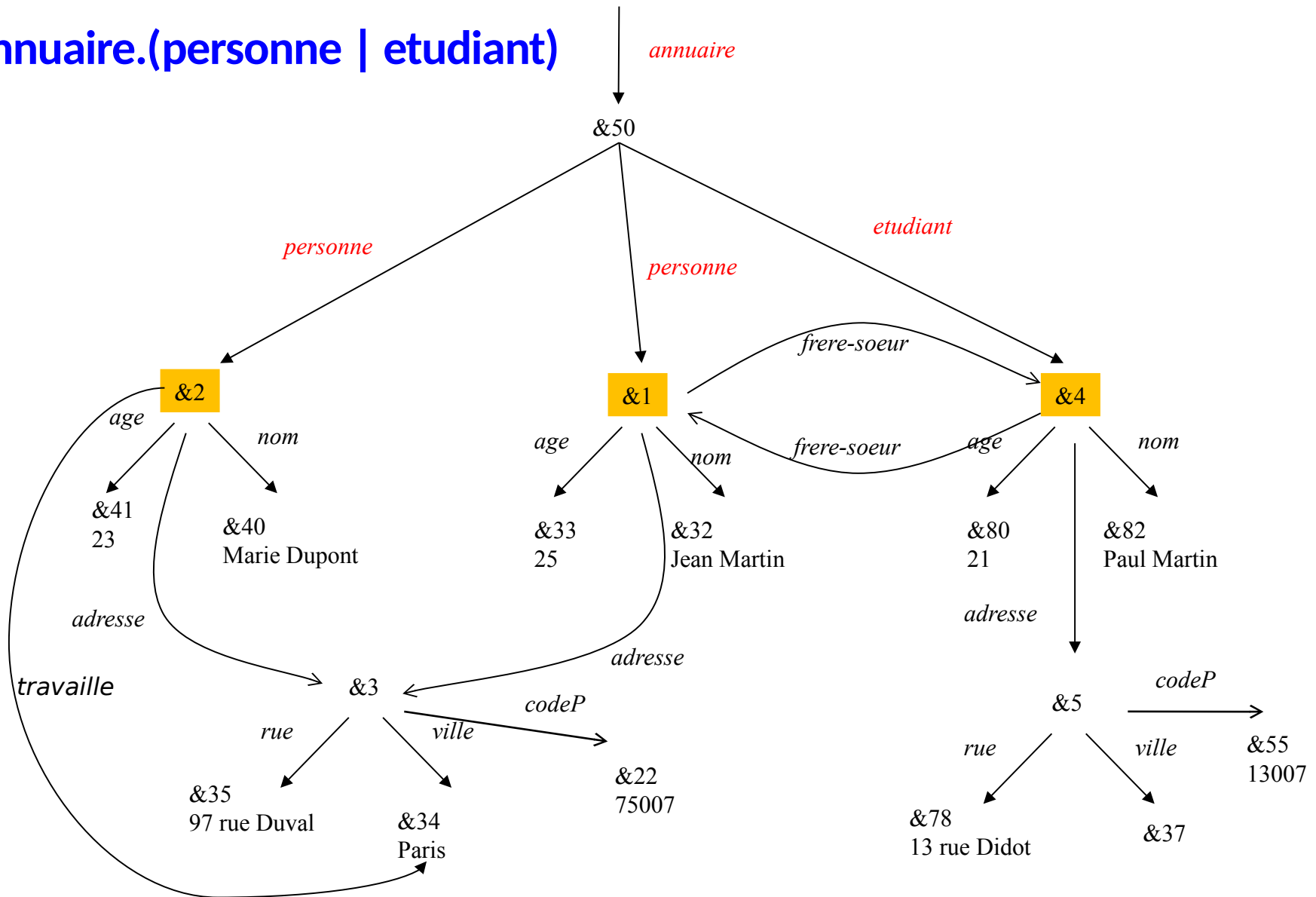
Expressions de chemin

Correspondance exacte	: personne
Correspondance partielle %	: per%
Alternative	: personne etudiant
Composition X.Y	: personne.nom
Arc optionnel (.X)?	: personne (.adresse)?.rue
Nombre arbitraire d'arcs (.X)*	: personne(.frere-soeur)*.nom
« joker » pour un nombre indéfini d'étiquettes non spécifiées #	: personne.#.ville

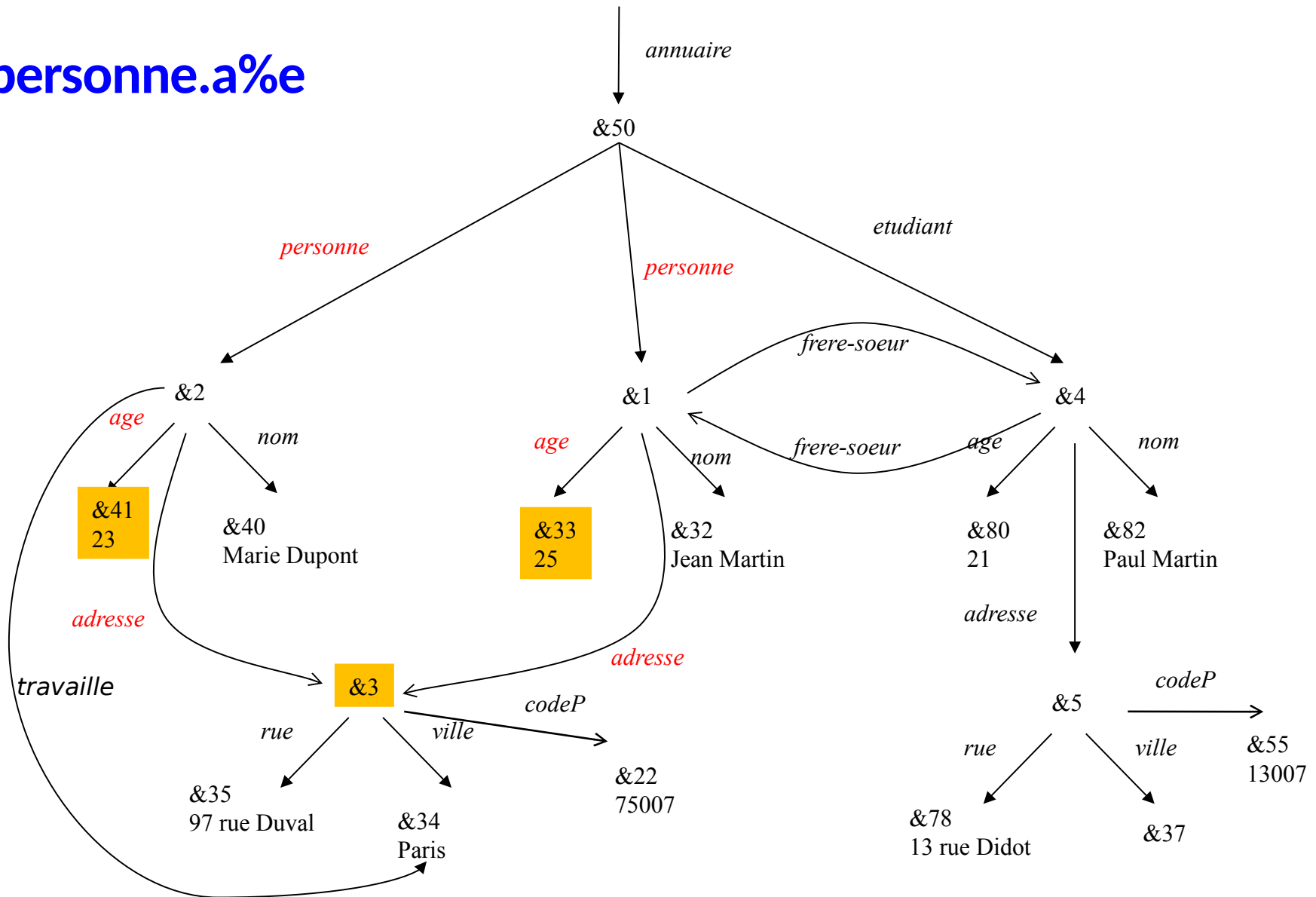
annuaire.personne



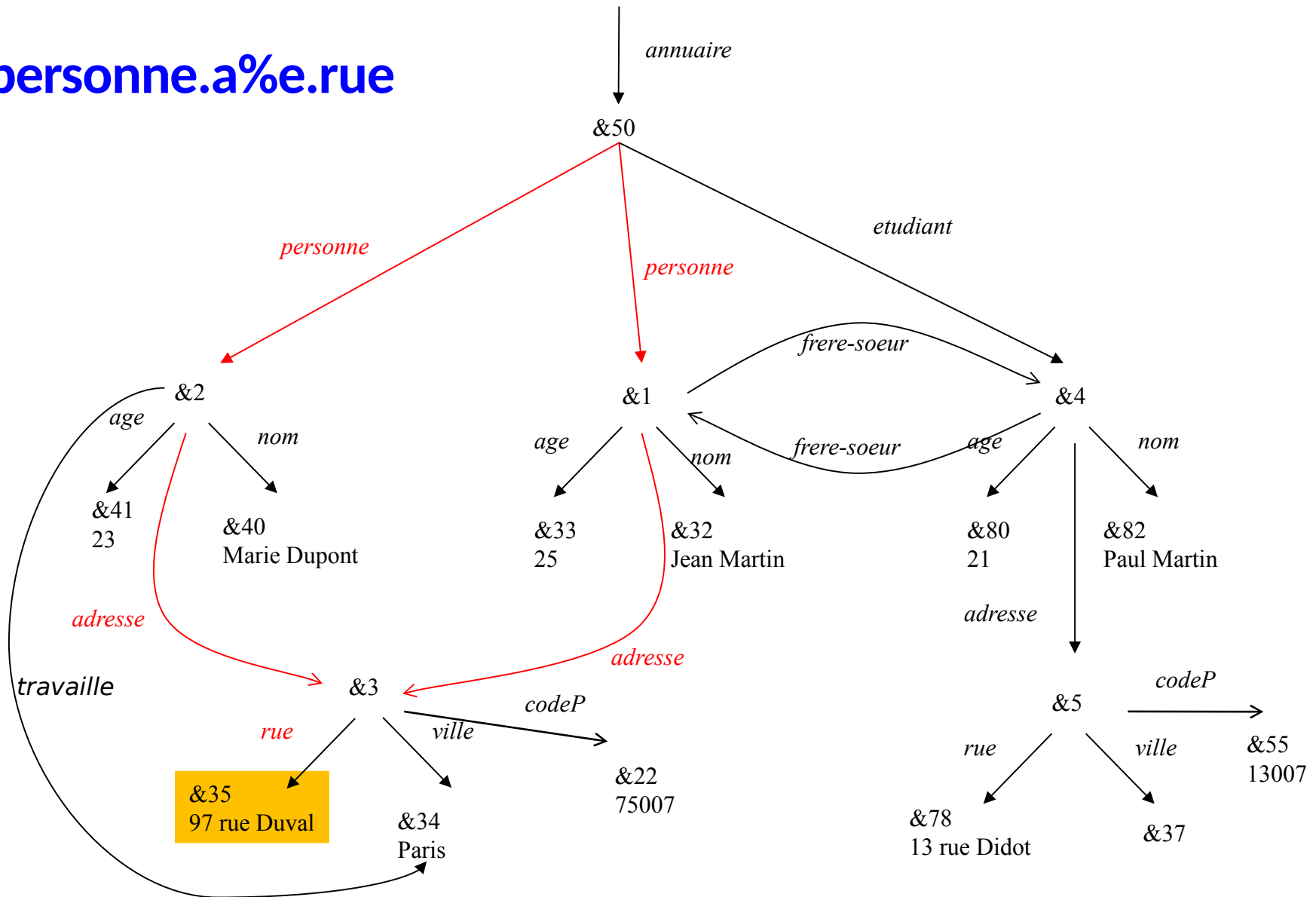
annuaire.(personne | etudiant)



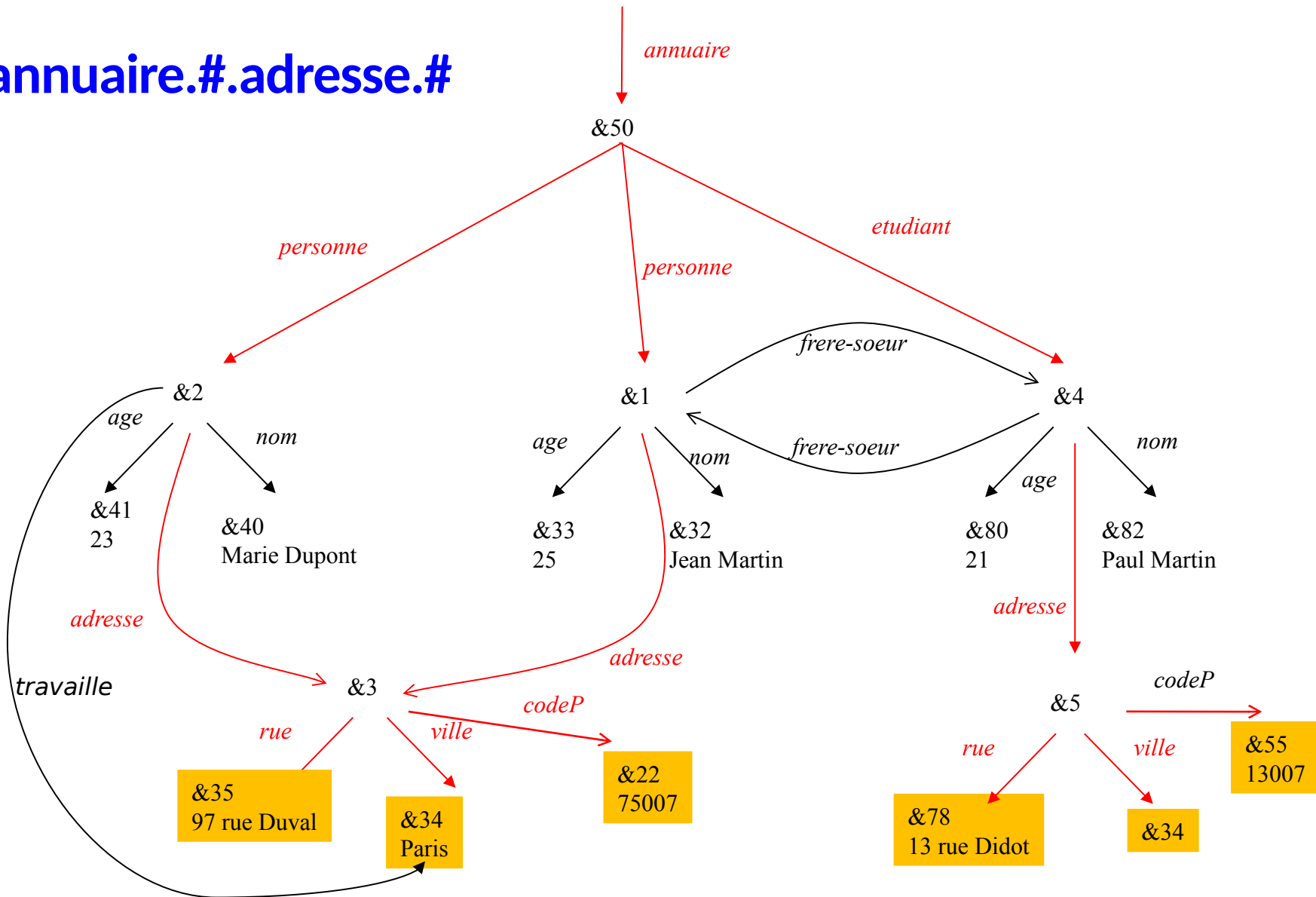
personne.a%



personne.a%e.rue



annuaire.#.adresse.#



Conclusion sur OEM

OEM : un des premiers modèles de données semi-structurées

Montre l'importance de pouvoir interroger les données et leur structure (« schéma »)

Modèle *auto-descriptif*:

- les étiquettes permettent de décrire l'information et de l'accéder
- schéma : description sans imposer des contraintes
- requêtes :
 - expressions de chemins sur les étiquettes
 - accès structuré sans connaissance à priori du schéma

De la théorie à la pratique ⇒ XML

XML

Extensible Markup Language

STANDARD DU W3C (1998)

LE LANGAGE XML : UNE SYNTAXE, UN FORMAT D'ÉCHANGE

DESCRIPTION DE DOCUMENTS XML : DTD (STRUCTURE TYPE DE DOCUMENT)- XSCHEMA

INTERROGATION : XPATH / XQUERY

XML

Objectifs

- Échange, partage, diffusion et publication de documents
- Recherche d'information : moteurs de recherche généralisée, portails spécialisés
- Commerce électronique : billetterie, catalogues électroniques,...

Format/langage standard pour les données semi-structurées du Web

- Consortium W3C (Oracle, IBM, MS, MIT, INRIA....) 1996
- Version 1.0 en 1998, nombreux développements depuis.
- Successeur de HTML, héritier de SGML

XML : Principes

Structure hiérarchique :

- un document XML est composé **d'éléments**, structurés en **hiérarchie**. Tout document possède **un élément racine**.

Balisage structurel :

- chaque élément est encadré par des balises (tag) ouvrante et fermante : **<cours>MLBDA</cours>**
- les noms des balises sont définies par les auteurs : séparer la structure logique des données de leur présentation (règles de transformation)

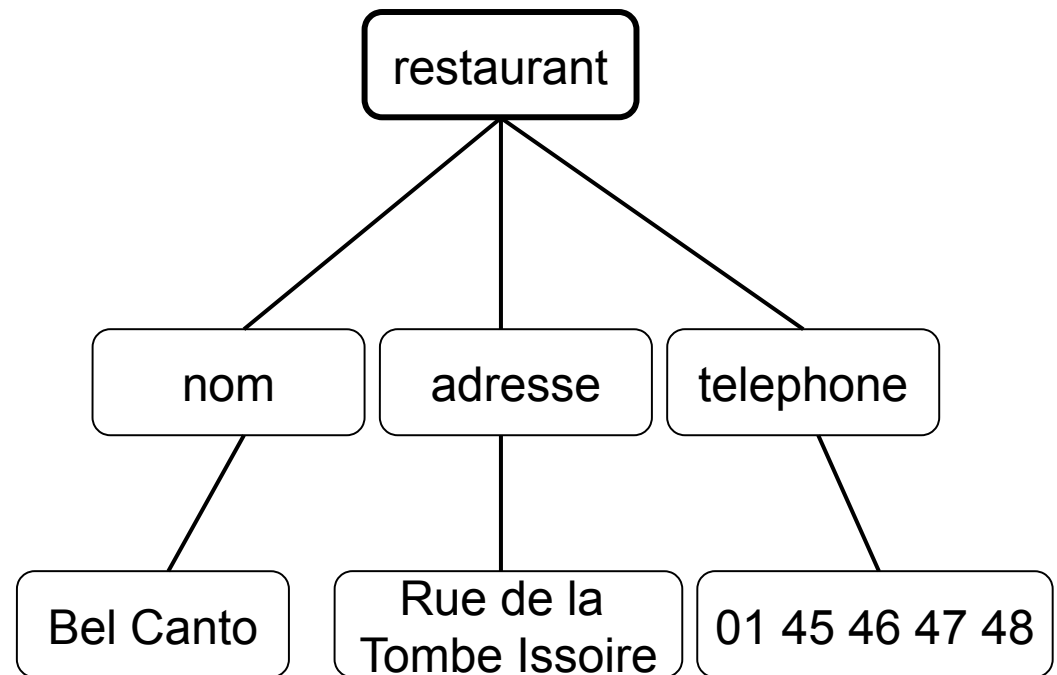
Forme sérialisée et forme arborescente

Un document XML peut se représenter sous deux formes:

- La **forme sérialisée** est la forme courante (contenu marqué par des balises). Elle est utilisée pour
 - Stocker un document dans un fichier
 - Echanger des documents
- La **forme arborescente** met en évidence la structure du document (facilite la conception des traitements).
 - Elle permet de spécifier des manipulations de données XML
 - Elle peut être utilisée par certaines applications qui gèrent les documents en mémoire (éditeurs XML).

Exemple

```
<restaurant>  
  <nom>Bel Canto </nom>  
  <adresse>  
    Rue de la Tombe-Issoire  
  </adresse>  
  <telephone>  
    01 45 46 47 48  
  </telephone>  
</restaurant>
```



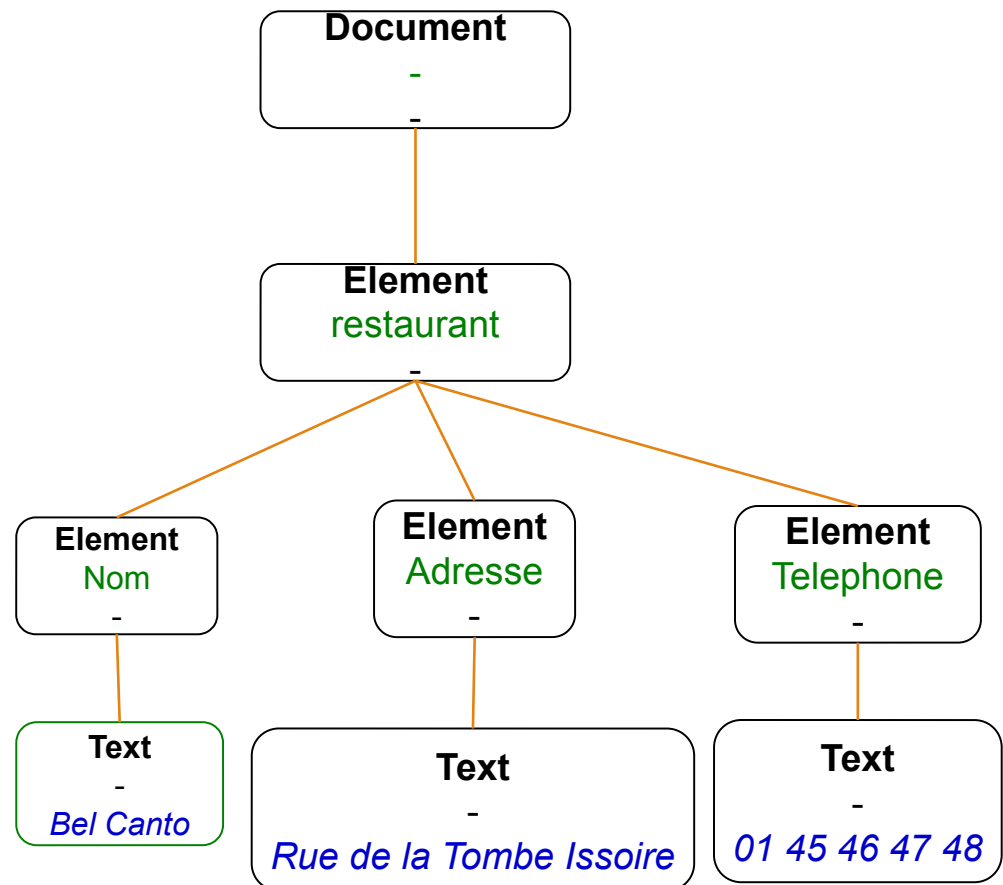
La structure des documents est définie par le *Document Object Model* (DOM)

Arbres DOM

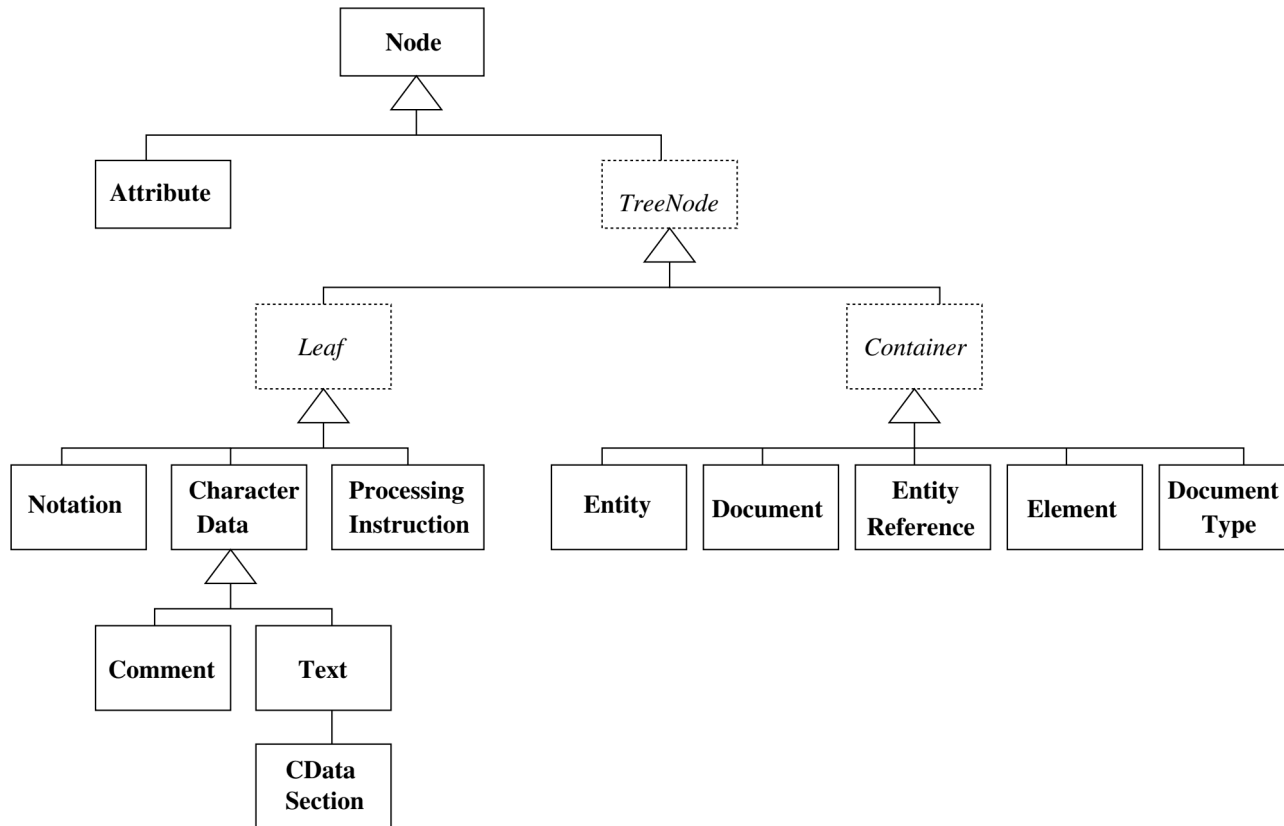
```
<restaurant>  
  <nom>Bel Canto </nom>  
  <adresse>  
    Rue de la Tombe-Issoire  
  </adresse>  
  <telephone>  
    01 45 46 47 48  
  </telephone>  
</restaurant>
```

Noeud DOM:

- **Type**
- **Nom** (optionnel)
- **Valeur** (optionnel)



Les Types DOM



Structure d'un document XML (syntaxe)

Un document XML est composé

- d'un **prologue** (*facultatif*)
- d'un **arbre d'éléments** (obligatoire : décrit le **contenu**)
- de **commentaires** et **instructions de traitement** (*facultatif*)

Un document peut être découpé en **entités** enregistrées dans un ou plusieurs fichiers.

Exemple

```
<?xml version="1.0"
      encoding='ISO-8859-1'
      standalone='yes'>
<personne>
  <nom>Martin</nom>
  <prenom>Jean</prenom>
  <adresse>
    <rue>rue Duval</rue>
    <ville>Paris</ville>
  </adresse>
<!-- COMMENTAIRE -->
</personne>
```

Déclaration XML (indications au processeur)

- xml version= "1.0" : décrit la version XML utilisée
- encoding='ISO-8859-1': codage de caractères utilisés dans le document
- standalone="yes" : existence de déclarations extérieures (yes = toutes les déclarations nécessaires au document sont incluses)

Le prologue peut contenir la déclaration d'une **DTD** qui décrit la structure du contenu (**voir plus loin**)

Arbre d'éléments (syntaxe)

Element : `<nom>contenu</nom>`

Un document est formé d'une **hiérarchie** d'éléments dénotant la structure du contenu.

- Tout élément fils est complètement inclus dans son père.
- L'élément racine est unique et contient tous les autres éléments.
- Un élément peut être vide ou contenir d'autres éléments, des données, des références à des entités, des sections littérales, des instructions de traitement.

Quelques règles syntaxiques:

- Tout élément doit avoir une balise ouvrante et une balise fermante.
- Un nom de balise doit commencer par une lettre, un `_`, ou `:` (pas de chiffre, de caractères réservés `?`, `/`, `%`, ...). Il ne doit pas contenir d'espace.
- On distingue les minuscules des majuscules

DTD : Définition de types de documents (Document Type Definition)

Une DTD est composée d'une suite de **déclarations** pour tous les composants utilisés dans un document XML :

- Éléments : `<!ELEMENT ...>`
- Attributs d'éléments : `<!ATTLIST ...>`
- Entités : `<!ENTITY ...>`
- Notations : `<!NOTATION ... >`

Elle peut aussi contenir des commentaires.

Elle peut être incluse dans le document (prologue) ou externe (standalone = 'no').

Déclaration d'Éléments

<!ELEMENT nom modèle_de_contenu>

Modèle de contenu :

- Expression régulières sur les noms des éléments fils

Expression régulières:

- Chaque nom d'élément est une expression régulière
- Si E1 et E2 sont des expressions régulières, alors
 - E1, E2 : concaténation
 - E1|E2 : choix
 - E1* : fermeture avec zéro ou plusieurs occurrences
 - E1+ : fermeture avec au moins une occurrence
 - E1? : option (0 ou 1 fois)

sont des expression régulières.

Exemple

```
<!ELEMENT adresse (numero, rue, ville)>  
<!ELEMENT coordonnées (portable|email)>  
<!ELEMENT enfants (personnes)*>  
<!ELEMENT texte-long (section+)>  
<ELEMENT section (titre?, (sous-section)*)>
```

Déclaration d'Éléments (suite)

#PCDATA Parsed Character Data :

- Contenu texte
- Ex: <!ELEMENT titre (#PCDATA)>

```
<!ELEMENT paragraphe (#PCDATA | em | exp | ind)*>  
<paragraphe>Du texte <em> en évidence </em>, 9 =  
3<exp>2</exp></paragraphe>
```

Exemple

Modèle de contenu mixte :

- (#PCDATA | élément1 | élément2 | ... | élémentn)*
- Pour mêler contenu textuel et éléments

Modèle de contenu vide : EMPTY

- Obligatoirement vide (infos uniquement dans attributs)
- Ne peut pas être composé (pas d'éléments fils)
- Syntaxe <element attribut=... />

Modèle de contenu libre : ANY

- Contenu quelconque (autres éléments et données)
- Sert à prototyper des DTD complexes

Attributs d'un élément

Les **attributs** sont un autre moyen de représenter l'information

- `<rapport langue='français' date='2014'>...</rapport>`

Un attribut est défini

- dans la balise ouvrante de l'élément et
- doit toujours avoir un nom et une valeur, encadrée par des apostrophes simples ou doubles.

L'ordre des attributs n'est pas important.

Il ne peut pas y avoir deux attributs de même nom dans un élément.

Les attributs ne sont pas ordonnés alors que les éléments le sont

Déclaration d'attributs

```
<!ATTLIST nom-élément nom-attribut1 type-attribut1 mode-défaut1  
    nom-attribut2 type-attribut2 mode-défaut2  
    .... >
```

- Type-attribut :
 - CDATA : la valeur de l'attribut est une chaîne de caractères
 - ID : identificateur d'élément, IDREF(S) : renvoi vers un (des) ID
 - NMTOKEN(S) : un ou des noms symboliques (sans blanc)
 - (a | b | c...) : type énumération de valeurs possibles
 - ENTITY(IES) : entités externes non XML
- Mode-défaut :
 - Valeur par défaut
 - Valeur constante : #FIXED
 - Présence obligatoire : #REQUIRED
 - Présence facultative : #IMPLIED

Exemples de déclarations d'attributs

DTD

```
<!ELEMENT date EMPTY >  
<!ATTLIST date format (ANSI | ISO | FR) #OPTIONAL  
          valeur #REQUIRED>
```

DOCUMENT

```
<date valeur='9 octobre 2019'></date>  
<date format='ISO' valeur='2019-10-9'/>
```

Exemples de déclarations d'attributs

DTD

```
<!ELEMENT ToDoList (task)*>  
<!ELEMENT task (#PCDATA)>  
<!ATTLIST task status (important|normal) "normal">
```

DOCUMENT

```
<ToDoList>  
  <task status="important">  
    This is an important task.  
  </task>  
  <task>  
    This is by default a task that has a normal status.  
  </task>  
</ToDoList>
```

Exemples de déclarations d'attributs

DTD

```
<!ELEMENT description (#PCDATA)>  
<!ATTLIST description xml:lang NMTOKEN #FIXED "en">
```

DOCUMENT

```
<description xml:lang="en">  
  The following section of code displays the menu of  
  user choices and gets the user's request. </description>
```

Attributs de type ID, IDREF(S)

Type ID :

- Permet d'identifier de façon unique un élément.
- L'unicité s'applique à tous les attributs de type ID du document.

Type IDREF :

- Permet de faire référence à un élément identifié par la valeur de l'attribut ID correspondant.
- La valeur d'un attribut de type IDREF doit correspondre à la valeur d'un attribut de type ID dans le document.

Type IDREFS :

- Permet de faire référence à plusieurs éléments identifiés par la valeur de l'attribut ID correspondant.
- Le séparateur est un espace (ex : att='p1 p2')

Exemple

```
<!ELEMENT DOC (AA)*>
<!ELEMENT AA (BB, CC)>
<!ELEMENT BB (#PCDATA)>
<!ELEMENT CC (#PCDATA)>
<!ATTLIST AA ident ID #REQUIRED
             refs IDREFS #IMPLIED>
<!ATTLIST BB num ID
             #REQUIRED>
<!ATTLIST CC ref IDREF
             #IMPLIED>
```

```
<DOC>
  <AA ident='10'>
    <BB num='1'>abc</BB>
    <CC ref='13'>cde</CC>
  </AA>
  <AA ident='99'>
    <BB num='2'>xxx</BB>
    <CC ref='10'>/>
  </AA>
  <AA ident='13' refs='1 99'>
    <BB num='3'>ttt</BB>
    <CC/>
  </AA>
  <AA ident='1'>
    <BB num='5'></BB>
    <CC ref=34/>
  </AA>
</DOC>
```

Référence invalide !

Entités

Caractère, chaîne de caractères, fragment, fichier externe...

- Déclarée par un nom : nom-entité et une valeur **val**
- Appelée dans le document par &nom-entité;
- Interprétation : remplacer &nom-entité; dans le document par **val**

Permet de réutiliser quelque chose défini ailleurs (macro, raccourci)

Plusieurs types d'entités :

- Entités internes : définies localement, comme une chaîne de caractères
- Entités externes : font référence à des fichiers externes
- Entités prédéfinies et entités caractères : référencent des caractères réservés en XML et des caractères qui ne sont pas sur le clavier

Entités prédéfinies et entités internes

Entités prédéfinies :

- amp (&), apos ('), quot ("), gt (>), lt (<)
- #code-unicode

Entités internes définies par l'utilisateur dans la DTD :

<!ENTITY nom-entité "valeur">

Exemple :

```
<!DOCTYPE mon-document [  
<!ENTITY copyright "&#x00A9; Editions UPMC."> ]>
```

```
<mon-document> &copyright; </mon-document>
```

Produira :

© Editions UPMC.

Entités externes et notations

Entités externes : inclure d'autres documents (formats) XML

```
<!ENTITY carte-de-visite SYSTEM "cdv.xml">
```

```
<message> ... &carte-de-visite; </message>
```

Notations : indiquer le traitement pour des entités externes

```
<!NOTATION gif SYSTEM "/usr/bin/xv">
```

```
<!ENTITY maphoto
```

```
SYSTEM "../mesphotos/mapomme.gif" NDATA gif>
```

```
<photo img='maphoto'>
```

- « Appel » dans un attribut
- La déclaration de notation (NDATA) indique le traitement utilisée

Entité paramètre

Entité paramètre : entité déclarée dans une DTD pour être utilisée dans cette DTD.

- Déclarée dans la partie interne (dans la partie DOCTYPE)

<!ENTITY % nom-entite "valeur entite" >

- La référence à l'entité (dans la DTD) se fait par %nom-entité.

Dans la DTD :

<!ENTITY %pub "Éditions ToutSavoir" >

<!ENTITY rights "Tous droits réservés" >

<!ENTITY book "J.Martin. Le Web et les BD, %pub. &rights">

Dans le document :

<p> &book </p>

On obtient :

<p>J. Martin. Le Web et les BD, Éditions ToutSavoir. Tous droits réservés.<p>

Rôles des DTD

Modèle selon une organisation hiérarchique (définition des éléments, attributs, contenus)

Spécifie la structure des instances de documents : cet élément contient ces éléments, ces attributs, etc.

Spécifie le type de données de chaque élément et attribut

Définition d'entités : mécanisme d'inclusion (interne, externe, paramètre) utile pour les opérations de modularisation et de réutilisation

Documents bien formés et documents valides

Un document XML est **bien formé** s'il respecte les règles syntaxiques définies par le standard XML :

- les éléments sont imbriqués (pas de chevauchement de balises)
- chaque attribut d'un élément a un nom unique,
-

Un document XML *bien formé* est **valide par rapport à une DTD** s'il respecte les déclarations de la DTD (éléments, entités, ...).

Exemple Document XML

DOCUMENTS XML

```
<?xml version="1.0" standalone="no" ?>
<!-- existence et adresse d'une DTD externe -->
<!DOCTYPE article SYSTEM "cours.dtd">
<article>
  <titre>Cours de M1.</titre>
  <nom IDCours='4IN801'>MLBDA</nom>
  <prerequis>
    <nom IDCours='3IN009'>BD</nom>
  </prerequis>
  <controle>examens répartis</controle>
  <contenu> ODMG. SQL3.XML.XPATH. </contenu>
</article>
```

COURS.DTD

```
<!ELEMENT article (titre*, nom, prerequis?, controle, contenu)>
<!ELEMENT prerequis (nom)>
<!ELEMENT titre (#PCDATA)>
<!ELEMENT nom (#PCDATA)>
<!ATTLIST nom IDCours ID #REQUIRED>
<!ELEMENT controle (#PCDATA)>
<!ATTLIST controle type-controle (examen | partiel) #IMPLIED>
<!ELEMENT contenu (#PCDATA)>
```

Limites des DTD

Syntaxe spécifique : une syntaxe pour les documents, une pour leur définition

Pas de possibilité de typer les contenus (types limités)

Typage faible des valeurs d'attributs

Les DTD ne sont pas suffisantes pour l'échange de données structurées (BD, commerce électronique, etc.).

⇒ **Schéma XML**